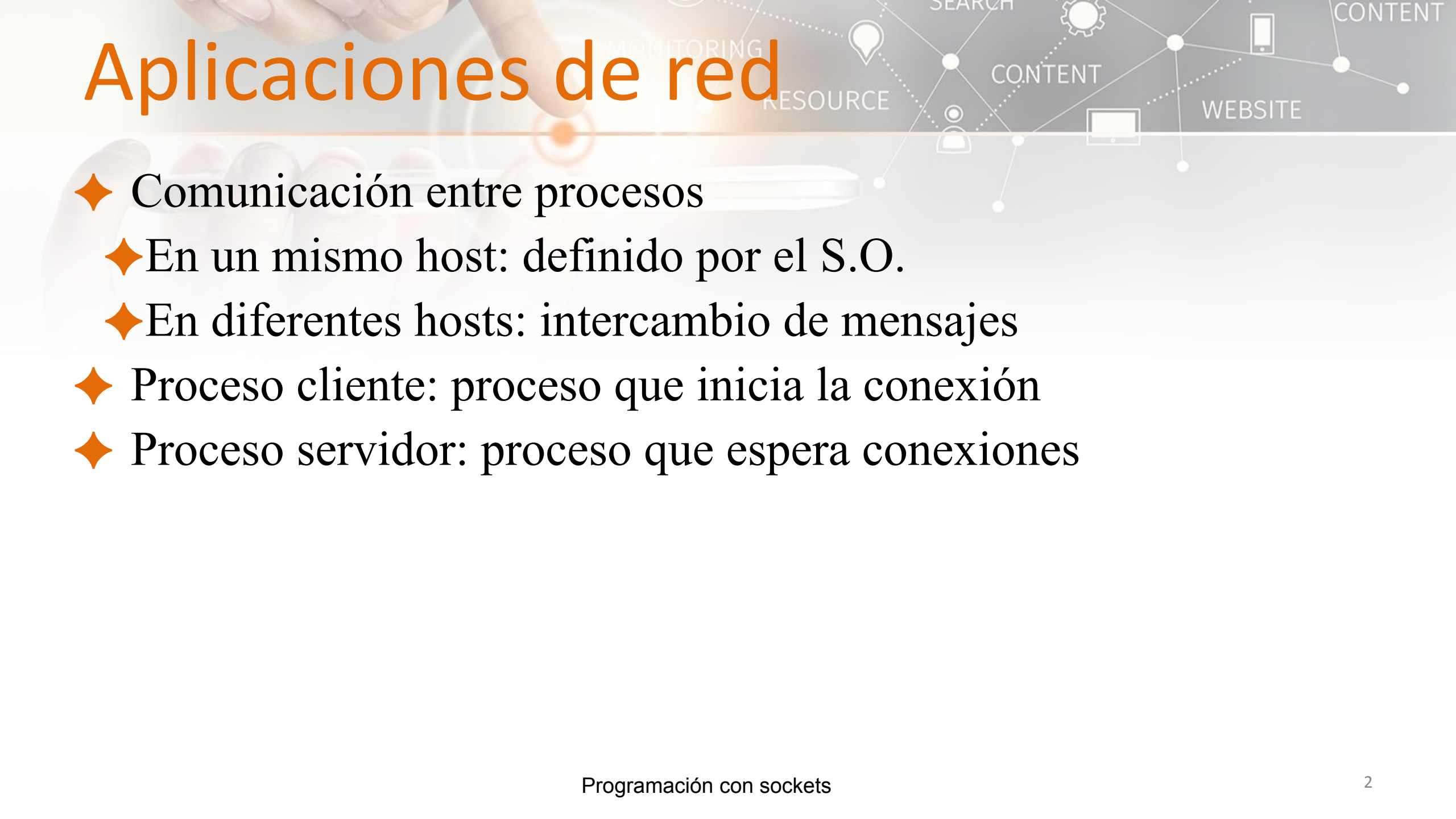




Sockets bloqueantes

Aplicaciones de red



- ◆ Comunicación entre procesos
 - ◆ En un mismo host: definido por el S.O.
 - ◆ En diferentes hosts: intercambio de mensajes
- ◆ Proceso cliente: proceso que inicia la conexión
- ◆ Proceso servidor: proceso que espera conexiones

Aplicaciones de red

- ◆ Se debe poder identificar los procesos
- ◆ Cada host se identifica por IP
- ◆ Muchos procesos corriendo en un host
- ◆ Para identificar un proceso se agrega un número de puerto
 - ◆ 80: HTTP server
 - ◆ 5432: PostgreSQL

Aplicaciones de red

- ◆ El protocolo de nivel de aplicación debe definir:
 - ◆ Tipos de mensaje que se intercambian
 - ◆ Sintaxis de los mensajes
 - ◆ Semántica de los mensajes
 - ◆ Reacción frente a errores

Interfaces de protocolos de transporte

El estándar TCP/IP sugiere que las API contemplen las siguientes operaciones básicas:

- ◆ Reservar los recursos locales para la comunicación
- ◆ Especificar *endpoints* origen y destino
- ◆ Iniciar una conexión (desde el cliente)
- ◆ Enviar un datagrama (desde el cliente)
- ◆ Esperar una conexión (desde el servidor)
- ◆ Enviar y recibir datos
- ◆ Poder determinar cuándo arriba información
- ◆ Generar datos urgentes
- ◆ Manejar el arribo de datos urgentes

Interfaces de protocolos de transporte

Sólo han tenido aceptación unas pocas API para los protocolos TCP/IP:

- ◆ socket API (Berkeley), diseñada para UNIX.
- ◆ Windows Socket (Microsoft).
- ◆ Transport Layer Interface TLI (AT&T), diseñada para System V UNIX.

Socket API

Berkeley sockets fue diseñado en los 80 para UNIX.

Utiliza system calls ya existentes cuando sea posible.

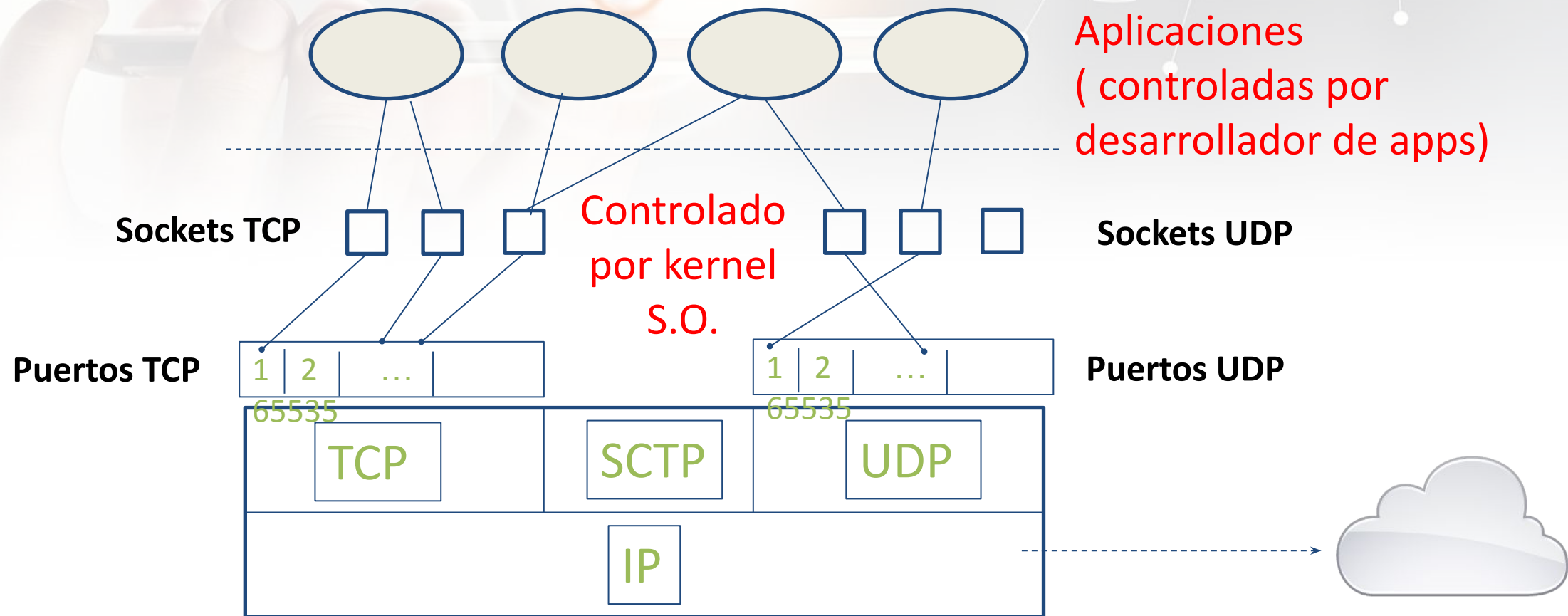
Provee funciones generales que soportan comunicaciones con distintos protocolos, no sólo TCP/IP.

Las funciones trabajan con *familias* de protocolos. Indicando el usuario qué tipo de familia (p.e. TCP/IP) y qué tipo de servicio requiere (con conexión o sin conexión).

Sockets: conceptos básicos

- ◆ El servidor debe estar en ejecución ANTES de que el cliente intente conectarse
- ◆ El servidor debe tener un socket (como una «puerta») abierto, por el cual recibir y enviar mensajes (segmentos)
- ◆ Los clientes necesitan un socket
- ◆ Los clientes necesitan conocer IP y número de puerto del socket del servidor

Aplicaciones de red : Sockets

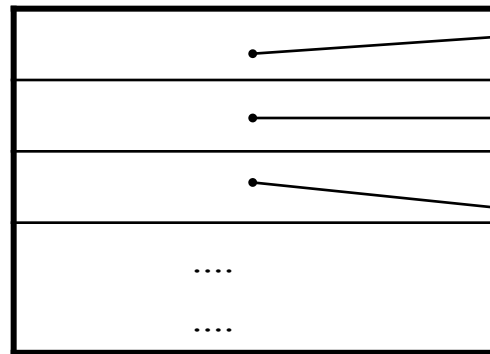


Socket API

Un proceso que necesita realizar operaciones de E/S utiliza el *system call* **open** que crea un *file descriptor*, el cual usa para acceder al archivo. El S.O. implementa *file descriptors* como un vector de punteros a estructuras de datos.

Espacio del usuario

tabla de descriptores
(una por proceso)



Recursos del kernel

Estructura de datos para *file id 0*

Estructura de datos para *file id 1*

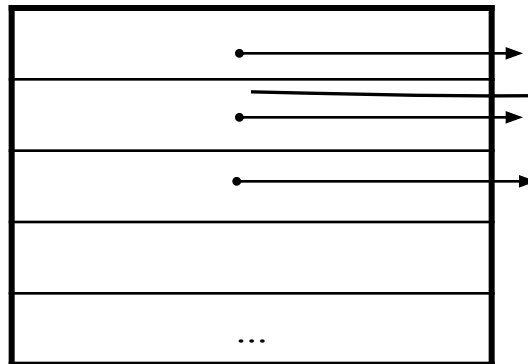
Estructura de datos para *file id 2*

Socket API

La API de sockets agrega una nueva abstracción para las comunicaciones en red, el *socket*. Al igual que con los archivos, cada socket es identificado por un *handle* y la información relacionada es almacenada en la tabla de *file descriptors*.

Espacio del usuario

tabla de descriptores
(una por proceso)



Recursos del kernel

family: PF_INET
service: SOCK_STREAM
Local IP:
Local port:
Remote IP:
...

Creación de un socket

```
#include <sys/socket.h>
```

```
// Returns: file (socket) descriptor if OK, -1 on error  
int socket(int domain, int type, int protocol);
```

domain: naturaleza de la comunicación y formato de la dirección

- AF_INET
- AF_INET6
- AF_UNIX
- AF_UNSPEC

Creación de un socket

```
#include <sys/socket.h>
```

```
// Returns: file (socket) descriptor if OK, -1 on error  
int socket(int domain, int type, int protocol);
```

type: caracteriza el tipo de comunicación

- SOCK_DGRAM
- SOCK_RAW
- SOCK_SEQPACKET
- SOCK_STREAM

Creación de un socket

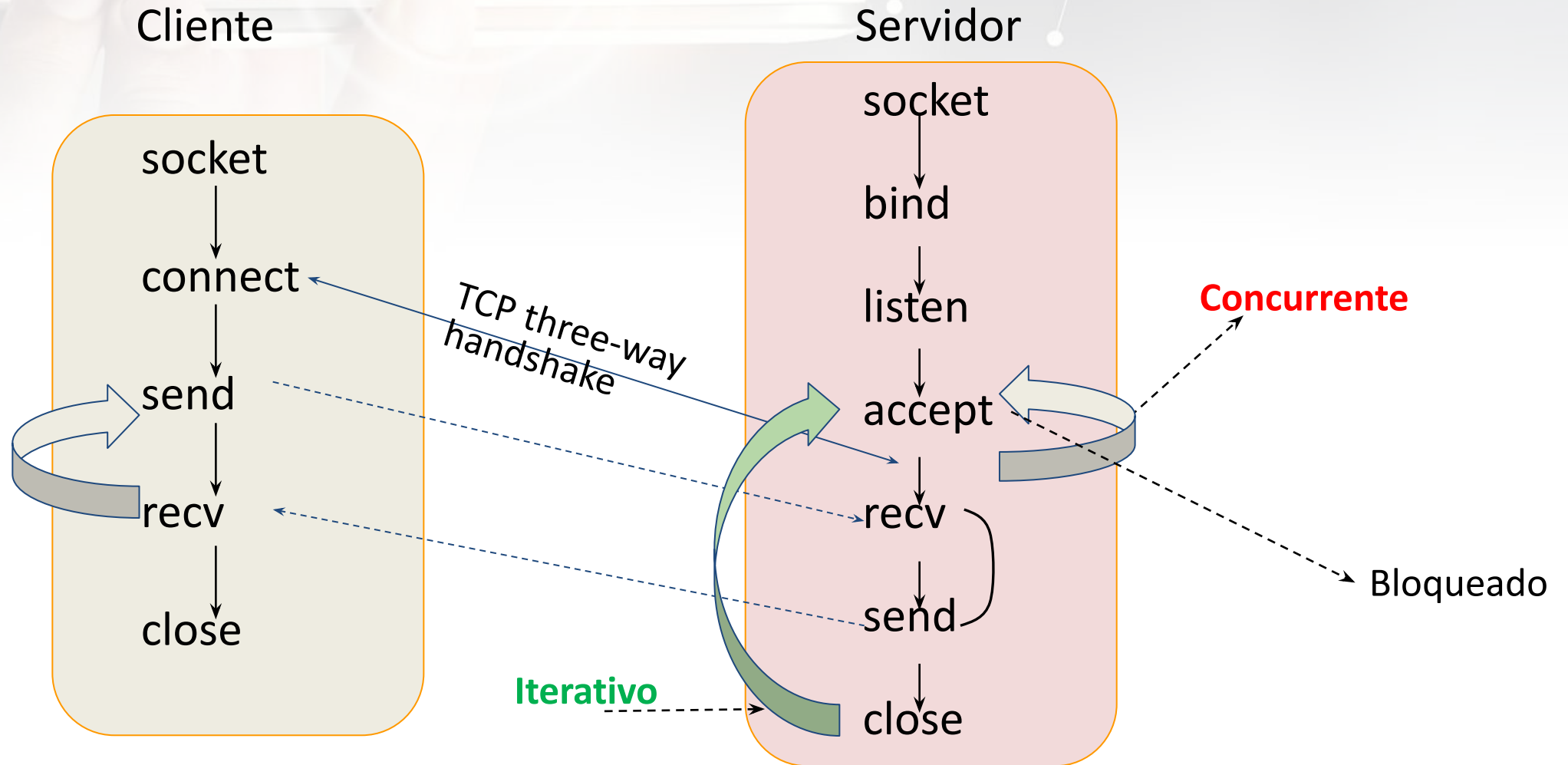
```
#include <sys/socket.h>
```

```
// Returns: file (socket) descriptor if OK, -1 on error  
int socket(int domain, int type, int protocol);
```

protocol: generalmente cero para que se seleccione el protocolo por defecto de acuerdo a *domain* y *type*

- IPPROTO_IP
- IPPROTO_IPV6
- IPPROTO_ICMP
- IPPROTO_RAW
- IPPROTO_TCP
- IPPROTO_UDP
- IPPROTO_SCTP

Socket API: ejemplo TCP



Diseño de servidores

Algoritmo para servidor iterativo no orientado a conexión

1. Crear un socket y ligarlo (*bind*) a un puerto
2. Leer un datagrama *request* de un cliente
3. Enviar datagrama/s como respuesta
4. Volver al punto 2

Un datagrama es un mensaje auto-suficiente

Diseño de servidores

Algoritmo para servidor iterativo orientado a conexión

1. Crear un socket y ligarlo (*bind*) a un puerto
2. Aceptar un pedido de conexión a través del socket
3. Obtener un nuevo socket para la conexión
4. Leer un *request* del cliente
5. Enviar una respuesta
6. Si el cliente no finalizó, volver al punto 4
7. Cerrar el socket creado en punto 3
8. Volver al punto 2

Socket I/O

La comunicación en un socket es bidireccional. Se puede deshabilitar I/O con *shutdown*

```
#include <sys/socket.h>

int shutdown(int sockfd, int how);
```

Posibles valores para *how*

- SHUT_RD
- SHUT_WR
- SHUT_RDWR

Formato de direcciones

Una dirección identifica un *socket endpoint*. El formato es específico para cada familia, y se castean a un formato genérico.

```
struct sockaddr {  
    sa_family_t sa_family;    // address family  
    char        sa_data[...]; // variable-length address  
    ...  
};
```


Formato de direcciones: ejemplo IPv4

```
struct in_addr {  
    in_addr_t s_addr;      /* IPv4 address */  
};  
struct sockaddr_in {  
    sa_family_t sin_family; /* address family: unsigned short */  
    in_port_t sin_port;     /* port number: uint16_t */  
    struct in_addr sin_addr; /* IPv4 address: uint32_t */  
    unsigned char sin_zero[8]; /* Linux: relleno */  
};
```

```
int bind(int socket, const struct sockaddr *address, socklen_t address_len);
```

```
struct sockaddr_in myAddress;  
...  
bind( socket, (struct sockaddr *) (&myAddress), sizeof(myAddress));
```

Formato de direcciones: ejemplo IPv6

```
struct in6_addr {
    uint8_t  s6_addr[16];      /* IPv6address */
};
struct sockaddr_in6 {
    sa_family_t    sin6_family; /* address family: unsigned short */
    in_port_t      sin6_port;   /* port number:  uint16_t */
    uint32_t       sin6_flowinfo; /* traffic class and flow info */
    struct in6_addr sin6_addr;   /* IPv6 address: */
    uint32_t       sin6_scope_id;
};
```

```
struct sockaddr_in6 myAddress;
...
bind( socket, (struct sockaddr *) (&myAddress), sizeof(myAddress));
```

Resolución de nombres

Un cliente debe especificar la dirección de un servidor utilizando la estructura `sockaddr_in`. ¿Qué sucede si el programa cliente conoce sólo el FQDN del servidor?

inet_addr: convierte de notación con puntos (“200.132.2.15”) a decimal.

gethostbyname: dado un string que contiene el FQDN de un host retorna una estructura que contiene –entre otras cosas- el IP del host en forma decimal.

getaddrinfo: más completa que la anterior.

```
#include <sys/socket.h>
#include <netdb.h>
```

```
int getaddrinfo(const char *restrict host,
               const char *restrict service,
               const struct addrinfo *restrict hint,
               struct addrinfo **restrict res);
```

```
void freeaddrinfo(struct addrinfo *ai);
```

Resolución de nombres

La estructura `addrinfo` contiene al menos los siguientes campos

```
struct addrinfo {  
    int      ai_flags;          // customize behavior  
    int      ai_family;         // address family  
    int      ai_socktype;       // socket type  
    int      ai_protocol;       // protocol  
    socklen_t ai_addrlen;       // length in bytes of address  
    struct sockaddr *ai_addr;    // address  
    char      *ai_canonname;     // canonical name of host  
    struct addrinfo *ai_next;    // next in list  
};
```

Resolución de nombres

Posibles valores del campo ai_flags

Flag	Description
AI_ADDRCONFIG	Query for whichever address type (IPv4 or IPv6) is configured.
AI_ALL	Look for both IPv4 and IPv6 addresses (used only with AI_V4MAPPED).
AI_CANONNAME	Request a canonical name (as opposed to an alias).
AI_NUMERICHOST	The host address is specified in numeric format; don't try to translate it.
AI_NUMERICSERV	The service is specified as a numeric port number; don't try to translate it.
AI_PASSIVE	Socket address is intended to be bound for listening.
AI_V4MAPPED	If no IPv6 addresses are found, return IPv4 addresses mapped in IPv6 format.

Figure 16.7 Flags for addrinfo structure

Ver addrinfo.c

Ejemplos TCP

- ◆ Cliente ECHO: `tcpEchoClient.c`
- ◆ Servidor ECHO iterativo: `tcpEchoServer.c`

Debug de procesos

Podemos usar comandos para depurar procesos, por ejemplo

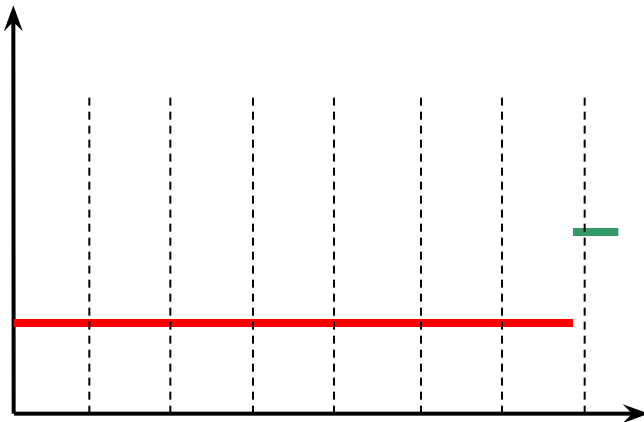
- ◆ truss en Solaris y FreeBSD
- ◆ strace en GNU Linux
- ◆ dtruss en OSX

Ejemplos UDP

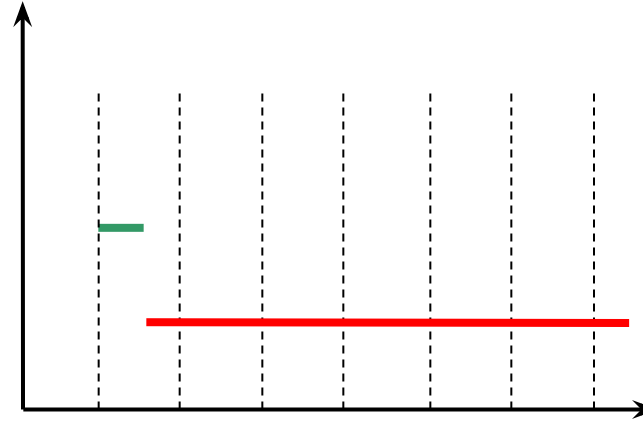
- ◆ Cliente ECHO: `udpEchoClient.c`
- ◆ Servidor ECHO: `udpEchoServer.c`

Servidores concurrentes vs iterativos

Ejemplo: un servidor ftp recibe un pedido para bajar un archivo de 200 MB e inmediatamente después otro pedido para bajar un archivo de 50 bytes.



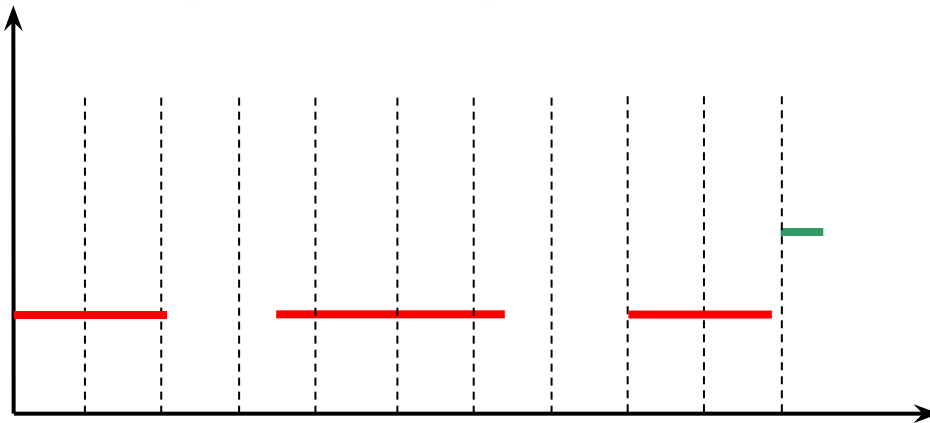
Servidor iterativo



Servidor
concurrente

Servidores concurrentes vs iterativos

El gráfico anterior contemplaba un uso ideal del “canal” al 100 %, y que el proceso no fuera bloqueado.



Servidor iterativo

Diseño de servidores: multitasking

Algoritmo para servidor concurrente orientado a conexión

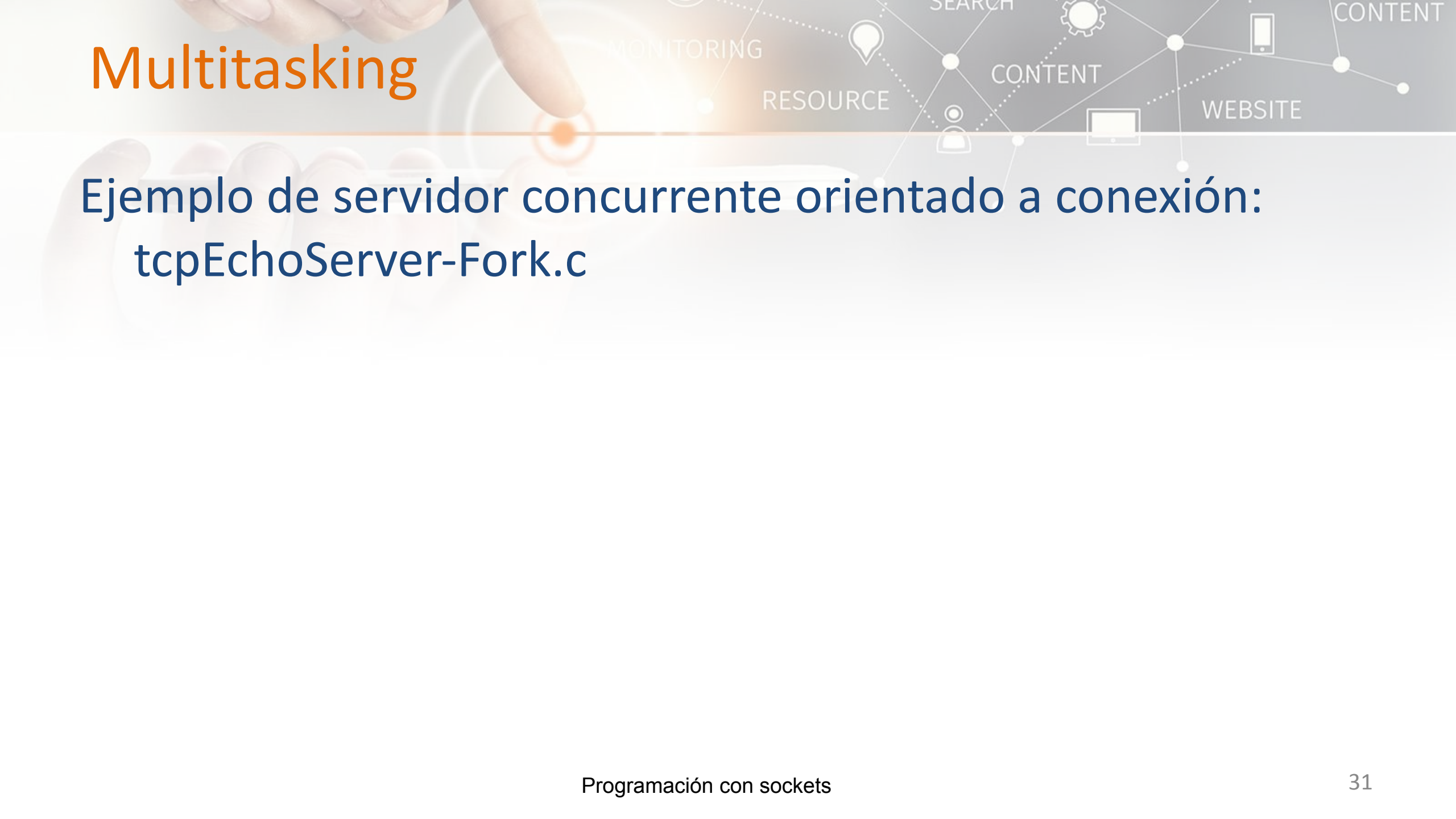
Master (Proceso Padre)

1. Crear un socket y ligarlo (*bind*) a un puerto
2. Aceptar un pedido de conexión a través del socket
3. Crear un proceso hijo para atender la conexión
4. Volver al punto 2

Slave (Proceso Hijo)

1. Atender la conexión pasada por el master
2. Interactuar con el cliente
3. Cerrar la conexión y finalizar el proceso

Multitasking



Ejemplo de servidor concurrente orientado a conexión:
`tcpEchoServer-Fork.c`

Multiplexing: select

- ◆ Todos los ejemplos vistos manejan I/O en un solo canal
- ◆ Un server puede manejar varios canales
 - ◆ escuchar en distintos puertos
 - ◆ sobre TCP, UDP, SCTP
 - ◆ superserver: un único programa para ECHO, Daytime, Chargen
- ◆ `select()` recibe una lista de descriptores y se bloquea hasta que alguno esté listo para I/O

Multiplexing: select

```
int select(int n,  
           fd_set *readfds,  
           fd_set *writefds,  
           fd_set *exceptfds,  
           struct timeval *timeout);
```

Cantidad máxima de file descriptors

Conjuntos de *fd* donde se marcará el que esté listo para read/write y condiciones “excepcionales”.

Tiempo máximo de bloqueo

Multiplexing: select

Los conjuntos de *fds* son un “array” de bits y existen macros para manipularlos

- ◆ `FD_ZERO(fd_set *set)`
- ◆ `FD_SET(int fd, fd_set *set)`
- ◆ `FD_CLR(int fd, fd_set *set)`
- ◆ `FD_ISSET(int fd, fd_set *set)`

Multiplexing: select

- Ejemplo (¿efectivo?) de servidor TCP ECHO con select
- TCPEchoSelect original.c extraído de <http://www.binarytides.com/multiple-socket-connections-fdset-select-linux/>

Multiplexing: pselect

Usar select implica un problema si queremos esperar a que o bien un file descriptor esté listo o recibamos una señal

```
int main(int argc, char *argv[])
{
    fd_set readfds;
    struct sigaction sa;
    int nfds, ready;

    sa.sa_handler = handler;      /* Establish signal handler */
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);
    /* ... */
    ready = select(nfds, &readfds, NULL, NULL, NULL);
    /* ... */
}
```

<https://lwn.net/Articles/176911/>

Multiplexing: pselect

pselect es similar a select pero además recibe un conjunto de señales que deberían ser bloqueadas durante la llamada a pselect

```
int pselect(int n, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,  
            const struct timespec *timeout, const sigset_t *sigmask);
```

<https://lwn.net/Articles/176911/>

Multiplexing: poll

A diferencia de select, la función poll recibe un vector que contiene únicamente los files id a "escuchar"

```
int poll (struct pollfd *fds, unsigned int nfds, int timeout);
```

```
struct pollfd {  
    int fd;  
    short events; // eventos que quiero vigilar  
    short revents; // eventos que realmente ocurrieron  
};
```

Ver ejemplo de uso y diferencias en

<https://devarea.com/linux-io-multiplexing-select-vs-poll-vs-epoll/#.Y0XLy2zMJhE>

y en el Drive

Multiplexing eficiente: epoll

```
int epoll_create(int size); // Deprecada a partir de Linux 2.6.7
int epoll_create1(int flags);
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);
int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);
```

```
struct epoll_event {
    uint32_t events; // EPOLLIN, EPOLLOUT, etc.
    union {
        void *ptr;
        int fd;
        uint32_t u32;
        uint64_t u64;
    } data;
};
```